I/O Techniques

Week 8 — Interrupts, Polling, Synchronous/Asynchronous Buses, DMA, and IOP

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-401: Computer Organization and Architecture

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 11, 2026

Where We Left Off (Week 7)

Last week: how the CPU controls *itself*

- Hardwired control (Week 6) and microprogrammed control (Week 7) both answered the same question: how does the CPU generate its own control-step sequence, cycle by cycle?
- Both designs assumed the CPU is the only thing that matters — registers, ALU, and an internal bus.

The gap Week 7 flagged

I/O devices — keyboards, disks, printers — are far slower than the CPU. Naively waiting for one wastes thousands of cycles. This week: how does the CPU decide what *outside* devices do, and when?

Roadmap: Four Questions, One Thread

1. **Program-controlled I/O (polling)** — what does the CPU do if it simply asks a device "are you ready?" in a loop?
2. **Interrupt-driven I/O** — what if the device asks the CPU instead?
3. **Synchronous vs. asynchronous buses** — one level down: how does the wire-level handshake between master and slave actually work?
4. **DMA and I/O processors** — what if the CPU is removed from the data path entirely?

Running thread for today

A single keyboard-input / display-output pair (`KIN`/`DOUT` status flags) threads through polling and interrupts; a 4096-byte block transfer threads through synchronous/asynchronous timing and into the DMA/IOP Act, where a 3-device arbitration example appears.

Today: who watches the device, and what happens when it's ready?

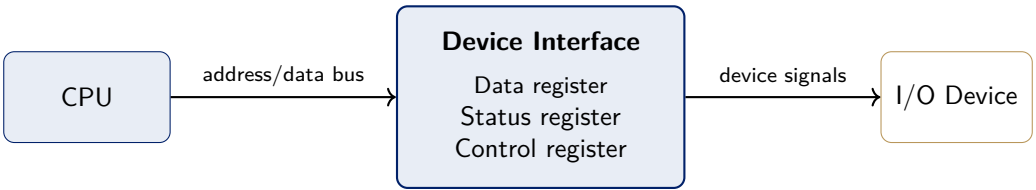
Socratic Question: The Speed Mismatch

Question

A CPU executes an instruction roughly every nanosecond. A keyboard produces one keystroke roughly every *hundred million* nanoseconds. If the CPU must read every keystroke, what should it do in between?

The simplest possible answer: give every I/O device a small set of registers the CPU can read and write *exactly like memory*. This is **memory-mapped I/O** — a device's data/status/control registers occupy addresses in the normal address space, no special I/O instructions needed [1].

The I/O Device Interface



Every interface supplies the same six functions: data/status/control registers, address decoding, timing generation, and format conversion — [1].

Definition: Program-Controlled I/O (Polling)

Polling

In **program-controlled I/O**, the CPU repeatedly reads a device's *status register* in a loop, checking a ready flag, until the device signals it is ready — then performs the data transfer itself.

- New notation: `KIN` — keyboard-input status flag, set when a keystroke is ready in `KBD_DATA`; `DOUT` — display-output status flag, set when the display is ready for the next character in `DISP_DATA`.
- The CPU is the only thing that ever asks; the device never initiates anything.

Worked Example: Polling Loops for KIN and DOUT

Keyboard input

READWAIT loop

```
READWAIT: Test KIN
          Branch=0 READWAIT
          MoveByte KBD_DATA, R
```

Display output

WRITEWAIT loop

```
WRITEWAIT: Test DOUT
           Branch=0 WRITEWAIT
           MoveByte R, DISP_DATA
```

Both loops re-execute the same three instructions on every pass — the CPU is spinning, doing no other useful work [1].

Socratic Check: What's Wasteful About Polling?

Question

A typist produces roughly 5 keystrokes per second. Between two keystrokes, how many `READWAIT` iterations does a 2 GHz CPU execute, and what is it accomplishing on each one?

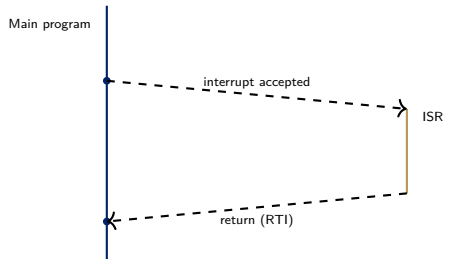
- On the order of hundreds of millions of iterations — every one of them re-reading a status flag that has not changed, accomplishing *nothing* useful.
- Polling is simple and requires no extra hardware, but it wastes CPU time in proportion to how much slower the device is than the CPU. Fine for a single slow device with nothing else to do; wrong at scale.

What if the device asked the CPU, instead of the CPU asking the device?

Definition: Interrupt

Interrupt

An **interrupt** is a signal from a device that suspends the CPU's current program, transfers control to a fixed handler — the **interrupt-service routine (ISR)** — and resumes the original program afterward, exactly where it left off.



The gap between the request and the jump is **interrupt latency**; registers the ISR must not disturb are saved in **shadow registers** [1].

Enabling and Disabling Interrupts

Two independent switches

- `IE` — a single interrupt-enable bit inside the processor status register (`PS`); when `IE`$= 0$ the CPU ignores *every* interrupt request, regardless of source.
- A per-device interrupt-enable bit in that device's control register (`KIRQ` for the keyboard, `DIRQ` for the display) — controls whether *this* device is allowed to request an interrupt at all.

A request reaches the CPU only when both switches agree: the device's own enable bit *and* `PS.IE` [1].

Worked Example: A 5-Step Interrupt-Handling Scenario

1. Device sets its status flag (e.g. `KIN`$= 1$) and, if `KIRQ`$= 1$, raises an interrupt-request line.
2. CPU finishes its *current instruction* (never mid-instruction), then checks `PS.IE`.
3. If enabled: CPU saves `PC` (and any registers the ISR will clobber) into shadow registers, disables further interrupts, and loads `PC` with the ISR's starting address.
4. ISR services the device (e.g. reads `KBD_DATA`, clearing `KIN`), then executes a return-from-interrupt instruction.
5. Return-from-interrupt restores the saved `PC` and re-enables interrupts — the interrupted program resumes with no memory of having been interrupted [1].

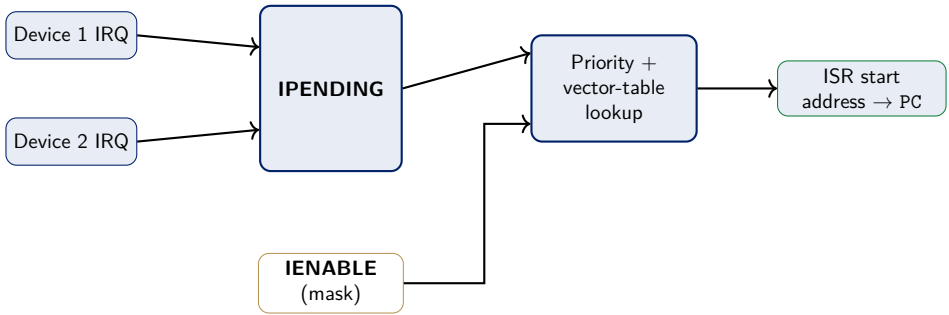
Handling Multiple Devices

Two designs for "which device interrupted?"

- **Polling the sources**: on any interrupt, the ISR checks each device's status flag in turn — simple, but the fixed check order costs time proportional to the number of devices.
- **Vectored interrupts**: each device supplies its own identifying code; hardware uses it to index directly into an **interrupt-vector table** — a table of ISR starting addresses, one entry per device — and jumps straight there.

Vectored interrupts trade a small amount of hardware (the vector table and the lookup) for constant-time dispatch, regardless of device count [1].

Interrupt Nesting and Priority



When both devices request simultaneously, the priority encoder grants the higher-priority one first; `IPENDING` keeps the other's request latched until it is serviced [1].

Socratic Check: Simultaneous Interrupt Requests

Question

Device 1 (higher priority) and Device 2 both request an interrupt on the same cycle. Device 1's ISR is still running when Device 2's request would normally be granted. What happens?

- If **nesting** is allowed and Device 2 outranks whatever priority level Device 1's ISR is running at, Device 2 can interrupt *that ISR* — the same 5-step scenario applies recursively.
- If nesting is disabled (or Device 2 is lower priority), Device 2's request stays latched in `IPENDING` until Device 1's ISR re-enables interrupts.

Polling vs. Interrupts: Trade-offs

	Polling	**Interrupts**
Who initiates	CPU (asks repeatedly)	Device (signals once, when ready)
CPU cost while waiting	Wasted spin cycles	Zero — CPU runs other code
Extra hardware	None	`IE`, per-device enables, vector table
Response latency	Depends on poll rate	Interrupt latency (usually smaller)
Best fit	One slow device, nothing else to do	Multiple/unpredictable devices

Same problem, opposite direction of initiative — interrupts move the waiting cost from the CPU to the hardware that detects readiness.

One level down: what does “the device is ready” look like on the wires?

Bus Structure: Master and Slave Roles

One shared bus, many devices

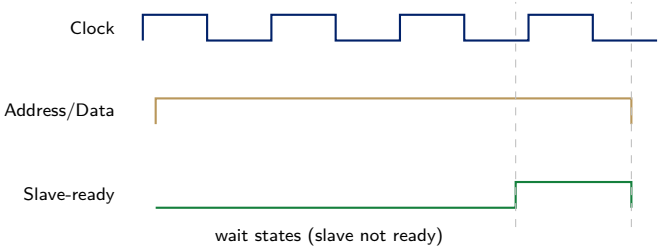
Every I/O interface in Act 1's diagram in fact hangs off a single shared **bus** — address, data, and control lines every device listens to. A **bus protocol** defines how a **master** (the device driving the transfer, usually the CPU) and a **slave** (the responding device) coordinate one data transfer without colliding.

This Act asks: when a master says "take this data" or "give me that data," how does the slave say "I'm not ready yet" or "here it is"? Two different answers: **synchronous** and **asynchronous** [1].

Definition: Synchronous Bus

Synchronous bus

All devices share a common **bus clock**. The master places an address/data on the bus at a fixed clock edge; every slave samples the bus at the *same* edge, with no per-transfer negotiation.

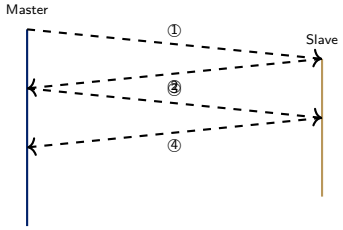


A slow slave stretches the transfer by holding `Slave-ready` low for extra clock cycles — the master simply waits, still synchronized to the shared clock [1].

Definition: Asynchronous Bus (Full Handshake)

Asynchronous bus

No shared clock. Master and slave exchange explicit **Master-ready**/**Slave-ready** signals, each one only changing *in response to* the other — a **full handshake**.



① Master-ready asserted
② Slave-ready (data valid)
③ Master-ready withdrawn (latched)
④ Slave-ready withdrawn

Each step waits for the previous one — **bus skew** (wires arriving at slightly different times) never causes a false sample, unlike a synchronous bus's single fixed clock edge [1].

Synchronous vs. Asynchronous: Trade-offs

	Synchronous	**Asynchronous**
Timing reference	Shared bus clock	Explicit handshake, no clock
Slow-device handling	Extra wait-state clock cycles	Handshake simply takes longer
Skew sensitivity	Sensitive — must beat clock period	Immune — self-timed
Per-transfer overhead	Low (one clock edge)	Higher (two round trips)
Best fit	Devices of similar, known speed	Mixed-speed / unpredictable devices

Same coordination problem, opposite mechanism — a shared clock everyone trusts, or an explicit conversation that trusts nothing.

Socratic Check: Why Not Always Use Asynchronous?

Question

Asynchronous handshaking tolerates skew and adapts automatically to any device speed. Why do most high-speed processor buses use synchronous protocols instead?

- Every asynchronous transfer pays *two full round-trip delays* (steps 1–2 and 3–4) — for devices that are consistently fast, this is pure overhead a synchronous bus does not pay.
- Synchronous buses only lose when a device is unusually slow (extra wait states) or the clock period must shrink to accommodate skew — exactly the trade-off the comparison table names.

Interrupts still cost one instruction per word transferred — what if the CPU left the loop entirely?

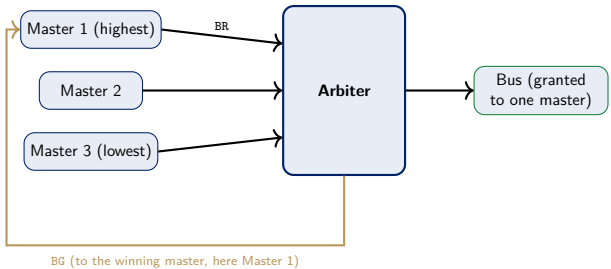
The Cost Interrupts Still Pay

Socratic question

A disk transfer moves a 4096-byte block, one 4-byte word at a time. With interrupt-driven I/O, the ISR executes on *every word*: read status, read/write data, update a memory pointer, decrement a counter, return. What does this cost at 4096 bytes, one interrupt per word?

$4096/4 = 1024$ ISR invocations for a *single* block transfer — interrupts removed the CPU's *waiting*, but not its involvement in *moving* the data. The next step removes the CPU from the data path itself.

Bus Arbitration: A Third Role for a Device



Any device connected via BR/BG can become bus **master**, not just the CPU — the arbiter grants the bus to exactly one requester, by priority, each time [1].

From Arbitration to Direct Memory Access

The functional idea, stated plainly

Once a device can become bus master, it can transfer data *directly to or from memory* — no CPU instruction touches the data at all. The CPU is interrupted *once*, when the whole block finishes, not once per word.

This edition of Hamacher's text (Ch. 7, arbitration) describes exactly this mechanism but never names it "DMA" — the term and its standard treatment below follow Stallings Ch. 8 [2], per the syllabus and `.claude/rules/textbook-grounding.md`.

Definition: Direct Memory Access (DMA)

DMA (general/standard treatment — Stallings Ch. 8)

A **DMA controller** is given a source/destination address and a word count by the CPU, then becomes bus master (via arbitration) and transfers the entire block directly to/from memory, interrupting the CPU only when the *block* — not each word — is complete [2].

- Pro: CPU involvement drops from "one ISR per word" — 1024 of them for our 4096-byte block — to "one setup, one completion interrupt" — a constant cost regardless of block size.
- Con: DMA controller and CPU now compete for the same memory bus (**cycle stealing**); extra hardware required.

Definition: I/O Processor (IOP)

IOP (general/standard treatment — Stallings Ch. 8)

An **I/O processor** generalizes DMA one step further: a dedicated processor, given a whole *channel program* (a short program describing several transfers, formats, and simple decisions), executes it independently — interrupting the main CPU only when the entire program completes, not per block.

Where DMA offloads *moving* data, an IOP offloads *managing* a sequence of I/O operations — the difference is how much decision-making is delegated away from the CPU, not just how the data moves.

Four I/O Techniques, Head to Head

	Polling	**Interrupts**	**DMA**	**IOP**
CPU cost / word	Every poll	Every word (ISR)	None (setup only)	None
CPU cost / block	—	Thousands of ISRs	One interrupt	One interrupt
Extra hardware	None	Vector table, `IE`	DMA controller	Dedicated processor
Best fit	One slow device	Multiple/sporadic devices	Bulk block transfer	Complex I/O subsystems

Same core question, four answers: who watches the device, and how much of the CPU does watching cost?

Socratic Check: Choosing a Technique

Question

A system reads one temperature sensor value per second (low rate, latency-tolerant) and separately copies a 10 MB file from disk to memory (high rate, throughput-critical). Which technique fits each?

- Temperature sensor: **interrupts** — rate is low, so ISR overhead per event is negligible, and the CPU is free the rest of the time.
- Disk copy: **DMA** — per-word CPU involvement at that data rate would dominate CPU time; DMA reduces it to one setup, one completion interrupt.

The Four Threads Meet

Every design choice moves the waiting cost somewhere else

- **Polling vs. interrupts (programmer's view):** polling wastes CPU cycles asking; interrupts let the device ask instead, at the cost of vector-table/priority hardware.
- **Synchronous vs. asynchronous (bus hardware):** a shared clock is cheap per transfer but skew-sensitive; a handshake is skew-immune but pays two round trips every time.
- **Interrupts vs. DMA/IOP (who moves the data):** interrupts still touch every word; DMA and IOP remove the CPU from the data path, trading extra hardware for near-zero per-word cost.

None of these is free — exactly the pattern Weeks 5–7 established for buses and control units, now applied to the world outside the chip.

Bridge to Week 9

From moving data to storing it efficiently

Weeks 1–8 built a CPU that computes, controls itself, and now moves data to and from slow devices. Week 9 asks: once data is in memory, how do we make *memory itself* fast enough to keep up with the CPU?

- DRAM is far slower than the CPU — the same speed-mismatch problem this week solved for devices reappears for memory itself.
- Cache memory, mapping functions, and replacement policies are Week 9's answer.

Summary

- **Memory-mapped I/O**: device registers (data/status/control) live in the normal address space; **program-controlled I/O (polling)** repeatedly tests a status flag (`KIN`, `DOUT`).
- **Interrupts**: the device signals readiness; `PS.IE` and per-device enables (`KIRQ`/`DIRQ`) gate delivery; **vectored interrupts** use `IPENDING`/`IENABLE` plus an interrupt-vector table for constant-time dispatch.
- **Synchronous bus**: shared clock, wait states for slow slaves. **Asynchronous bus**: `Master-ready`/`Slave-ready` full handshake, skew-immune but higher per-transfer overhead.
- **Bus arbitration** (`BR`/`BG`) lets a device become bus master — the functional basis for **DMA** (block transfer, one completion interrupt) and **IOP** (whole channel programs, general/standard treatment per Stallings Ch. 8).
- **Four techniques, one trade-off axis**: how much of the CPU does watching and moving the data cost, and how much hardware buys that cost down.

References

V. Carl Hamacher, Zvonko G. Vranesic, Safwat G. Zaky, and Naraig Manjikian.
Computer Organization and Embedded Systems.
McGraw-Hill, 6th edition, 2012.
ISBN 978-0-07-338065-0. Bib key retained as Hamacher2002_computer_organization for citation-key stability across existing lectures; the file indexed under master_supporting_docs/CS401/supporting_books/Hamacher2002/ and page-cited by Week 8 is this 6th edition, not the 5th ed. (2002) the key name suggests – see that folder's index.md Edition notice, 2026-08-11.

William Stallings.
Computer Organization and Architecture: Designing for Performance.
Pearson, 10th edition, 2015.